

Database Administration

José Orlando Pereira

Departamento de Informática
Universidade do Minho



Query processing

a
2
3

"select a from X natural join Y where c = 3;"

X

a	b
1	aaa
2	bbb
3	ccc

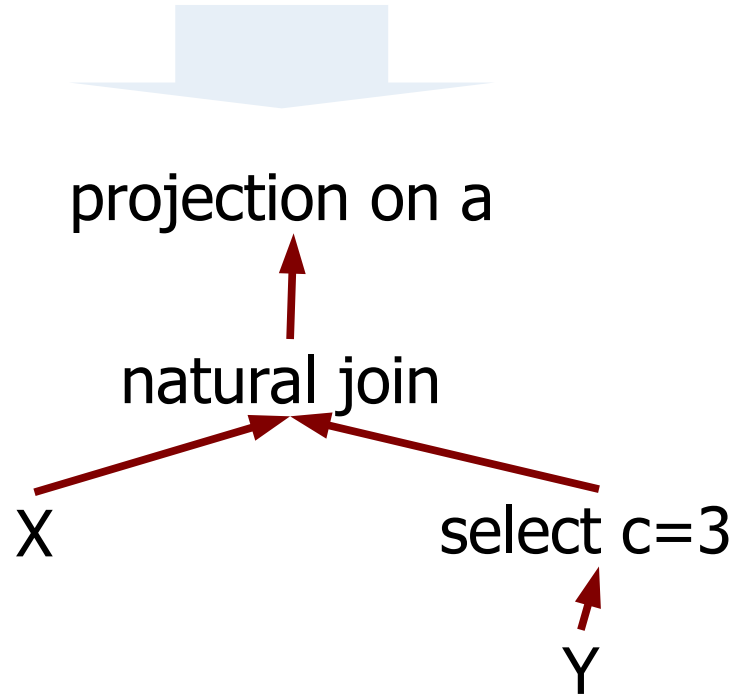
Y

b	c
aaa	1
bbb	2
bbb	3
ccc	3
ddd	4

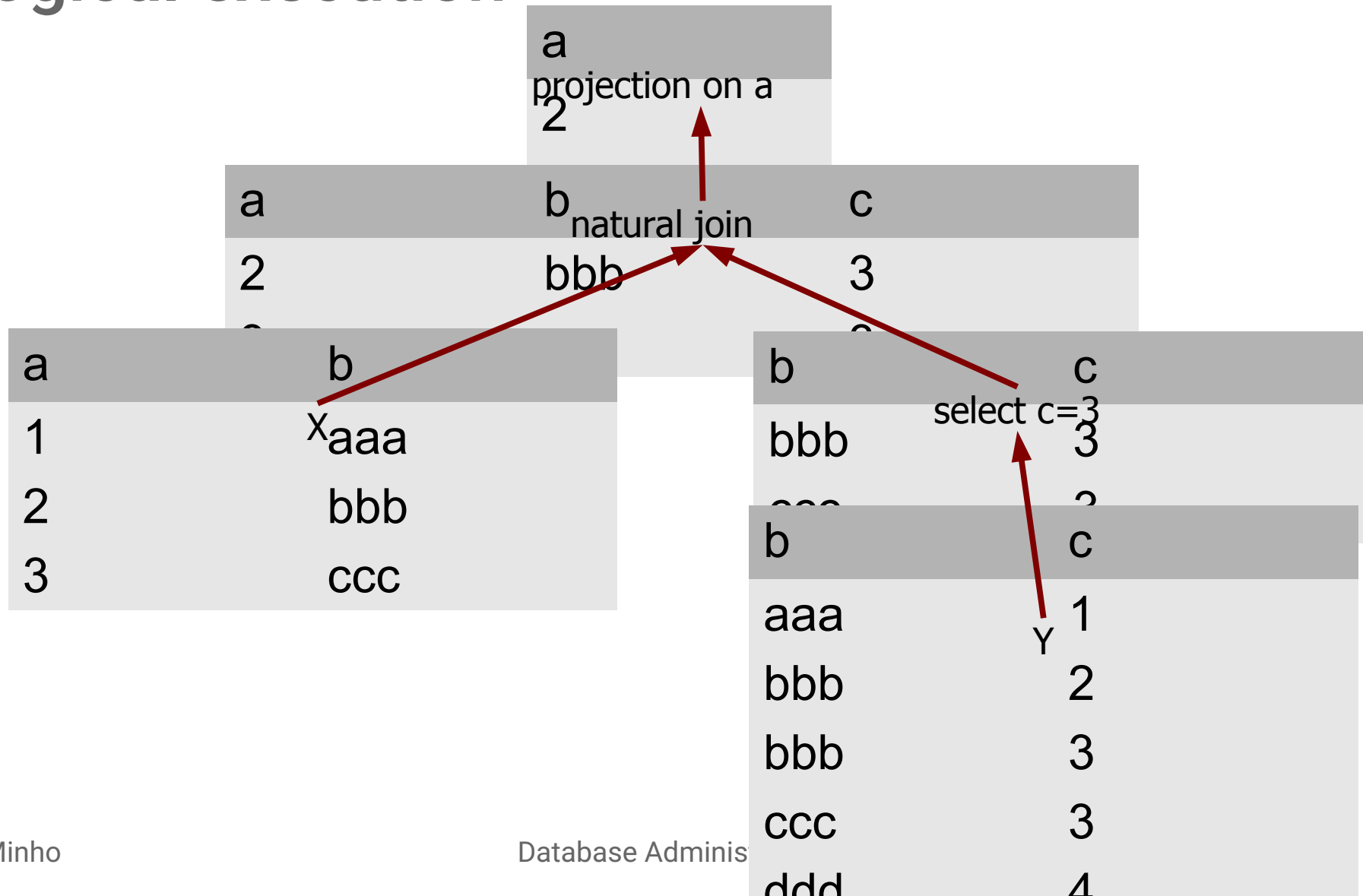
Translation + validation

SQL { "select a from X natural join Y where c = 3;"

Relational
algebra {



Logical execution



Roadmap

- What physical operators exist for each logical operation?
- How are physical operators implemented and composed?
- Later: How are physical operators selected?

One-pass, record-at-a-time

- Operators:
 - Sequential scan
 - Selection
 - Projection
- Memory requirements:
 - No more than one record required
 - Always possible

User defined functions (UDFs)

- Functions can be defined in various languages
 - Python
- Scalar functions used in projections/selections:
`SELECT a, f1(a) FROM t;`
`SELECT * FROM t WHERE f2(a)`
- Table functions can be used in sequential scans:
`SELECT * FROM f3(...);`
- User defined functions can access external services:
 - Web services
 - GenAI

One-pass, full relation, unary

- Duplicate elimination:
 - Cache unique records
 - “select distinct * from X;”
- Grouping and aggregation:
 - Cache groups
 - “select count(*) from X group by b;”
- Sorting:
 - Cache all records and sort in memory
 - “select * from X order by b;”

One-pass, full relation, binary

- Union, difference, intersection, product, join:
 - Read and cache the smallest relation
 - Organize for fast look-up (e.g. hash)
 - Read and operate on each record from the largest relation

One-pass, full relation, binary

- Load smaller table into memory and add search structure:

a	b	c
2	bbb	3
3	ccc	3

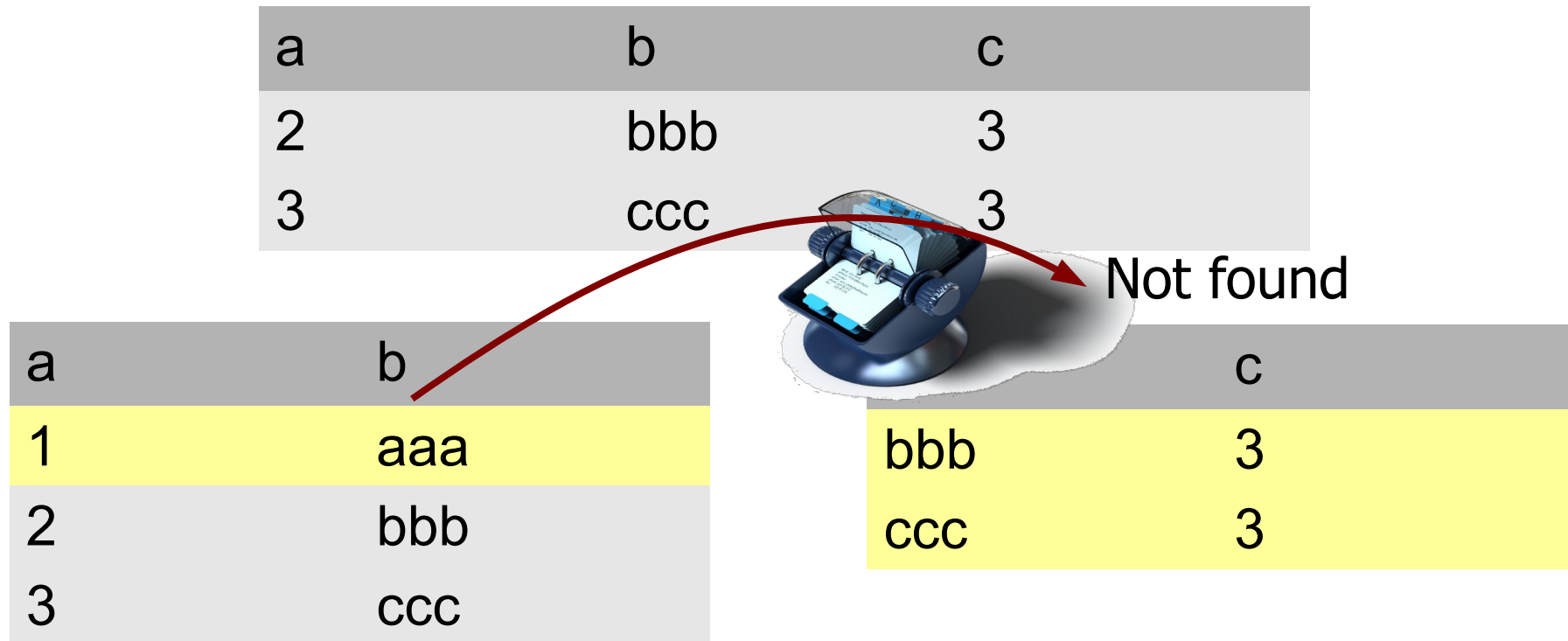
a	b
1	aaa
2	bbb
3	ccc



c
bbb 3
ccc 3

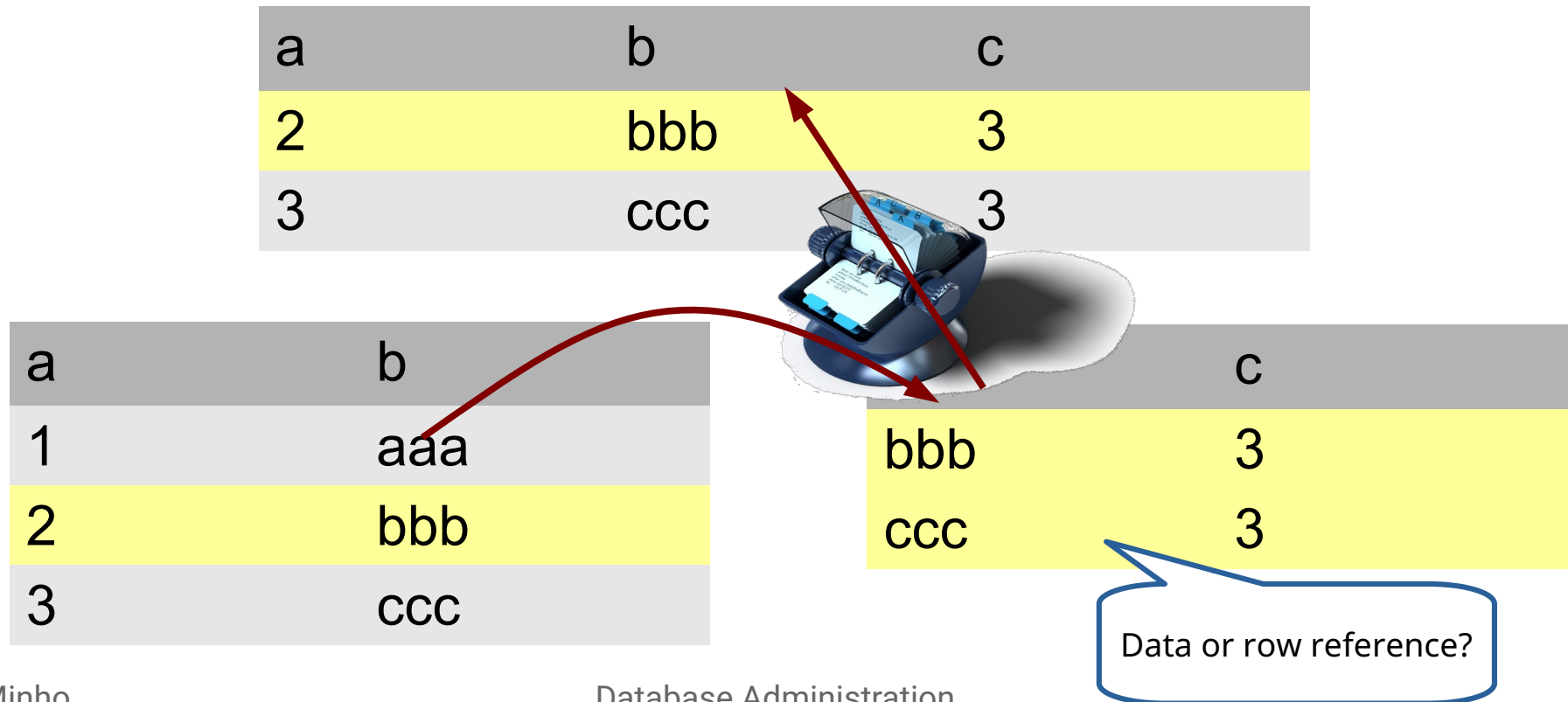
One-pass, full relation, binary

- Test each record from the largest relation:



One-pass, full relation, binary

- Test each record from the largest relation:



Nested-loop join (NLJ)

a	b
1	aaa
2	bbb
3	ccc

b	c
bbb	3
ccc	3
b	c

a	b
1	aaa
2	bbb
3	ccc

b	c
bbb	3
ccc	3
b	c
bbb	3
ccc	3

Nested-loop join (NLJ)

- Memory requirements:
 - One record from each relation
- Operations:
 - If outer loop has N records
 - Reads inner relation N times

Block-based NLJ

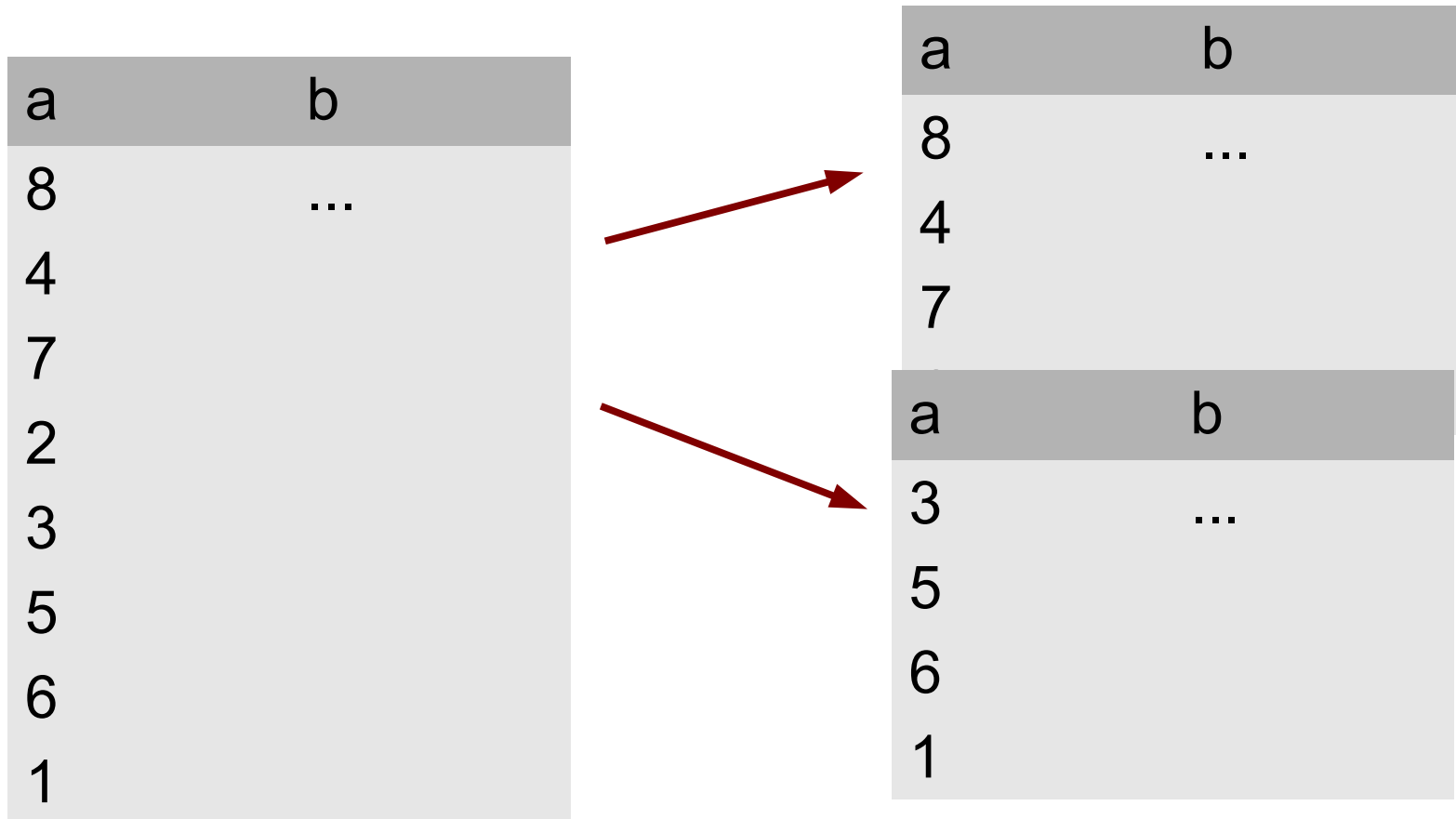
- Much smarter: Execute NLJ by blocks
- Memory requirements:
 - One block from each relation
- Operations:
 - If outer loop has N records / B blocks ($B \ll N$)
 - Reads inner relation B times ($B \ll N$!)

Large relations and sorting

- Algorithms using sorted data are more efficient (e.g. than nested loops)
- How to sort data that does not fit in memory?

Merge-sort

- Split data in chunks that fit in memory:



Merge-sort

- Load and sort each of them:

a	b
2	...
4	
7	
8	

a	b
3	...
5	
6	
1	

Merge-sort

- Load and sort each of them:

a	b
2	...
4	
7	
8	

a	b
1	...
3	
5	
6	

Merge-sort

- When iterating, select the next record from the fragment with the next key:

a	b
2	...
4	
7	
8	

a	b
1	...
3	
5	
6	

Merge-sort

- When iterating, select the next record from the fragment with the next key:

a	b
2	...
4	
7	
8	

a	b
1	...
3	
5	
6	

Merge-sort

- When iterating, select the next record from the fragment with the next key:

a	b
2	...
4	
7	
8	

a	b
1	...
3	
5	
6	

Two-pass, full relation, unary

- First pass is sorting
- Duplicate elimination:
 - Cache last record
 - “select distinct * from X;”
- Grouping and aggregation:
 - Cache last group
 - “select count(*) from X group by b;”

Two-pass, full relation, binary

- Union, difference, intersection, product, join:
 - Read record R1 from sorted relation T1
 - Read record R2 from sorted relation T2
 - If $R1 = R2$:
 - Use R1 and R2
 - If $R1 < R2$:
 - R1 does not exist in T2
 - Skip R1
 - If $R2 < R1$
 -